# Laporan Benchmark: ACS vs GenieACS

Beban simulator CPE identik pada kedua sistem · Kamis, 22 Oktober 2026

| | |
|---|---|
| **Sistem dibandingkan** | **ACS kita** (Go + MariaDB + Redis, cabang `dev`, termasuk optimasi hari ini) vs **GenieACS 1.2.16** (Node.js + MongoDB 6, konfigurasi bawaan) |
| **Beban** | 1.500 perangkat simulasi (profil ZTE TR-098), 64 sesi paralel, 1 registrasi + 4 putaran Inform periodik + 500 tugas `GetParameterValues` |
| **Lingkungan** | Satu laptop (8 CPU, 16 GB RAM, Docker 29.8.0 di Windows) yang menjalankan kedua ACS *dan* generator beban; kedua stack dibangun baru dengan database kosong, lalu diuji bergantian (ACS kita dulu, GenieACS kemudian) |
| **Alat & cara ulang** | `cmd/acsbench`, `scripts/bench.sh --devices 1500 --concurrency 64 --rounds 4 --tasks 500` (±10 menit); hasil mentah `bin/bench-*.json` |

## 1. Kesimpulan

**Pada beban simulator ini, ACS kita setara hingga lebih cepat dari GenieACS pada jalur Inform, dengan memori proses jauh lebih kecil. Belum ada bukti keunggulan pada fase tugas, dan belum terbukti pada perangkat nyata.**

Setelah pemanasan, ACS kita menangani 530–783 sesi/detik (GenieACS 437–504) dengan p95 116–244 ms (GenieACS 239–286 ms). Proses ACS memakai ±25 MiB memori (GenieACS ±589 MiB). Pada fase tugas keduanya setara: GenieACS sedikit lebih tinggi throughput-nya, ACS kita sedikit lebih baik di ekor latensi. Tidak ada galat pada kedua sistem.

| 783 | 116 ms | 25 MiB | 0 |
|---|---|---|---|
| sesi/dtk ACS kita, putaran 4 (GenieACS 449) | p95 ACS kita, putaran 4 (GenieACS 280 ms) | memori proses ACS kita (GenieACS 589 MiB) | galat dari 8.000 sesi per sistem |

**Batas klaim (baca dulu).** Beban berasal dari **simulator** CPE, bukan firmware nyata, dan semuanya berjalan di **satu laptop** yang juga menjalankan generator beban; stack yang tidak sedang diuji tetap hidup (idle), sehingga hasil dipengaruhi kondisi mesin. Angka di bawah adalah **satu kali run penuh**; pada pengukuran ulang di hari yang sama, fase tugas ACS kita bergeser antara 79 dan 143 sesi/detik, jadi selisih kecil (≤±30%) tidak boleh dibaca sebagai pemenang. Konfigurasi bawaan dipakai di kedua sisi, dan **kompatibilitas ONT tidak dibuktikan oleh benchmark ini**.

## 2. Metode

- **Beban identik.** Alat yang sama (`cmd/acsbench`) mengirim urutan pesan CWMP yang sama ke kedua sistem. Tiap pekerja memakai satu koneksi TCP persisten, dan satu sesi CWMP selalu berada di satu koneksi seperti CPE nyata.
- **Gaya pesan.** SOAP ditulis dengan awalan namespace (`<soap-env:…>`) seperti CPE nyata. Tanpa itu GenieACS menolak Inform ("Unrecognized CWMP version"); ACS kita menerima kedua gaya.
- **Fase.** (1) registrasi `0 BOOTSTRAP`; (2) empat putaran Inform periodik `2 PERIODIC`; (3) fase tugas: antre `GetParameterValues` lewat API, CPE menjawab dalam sesinya.

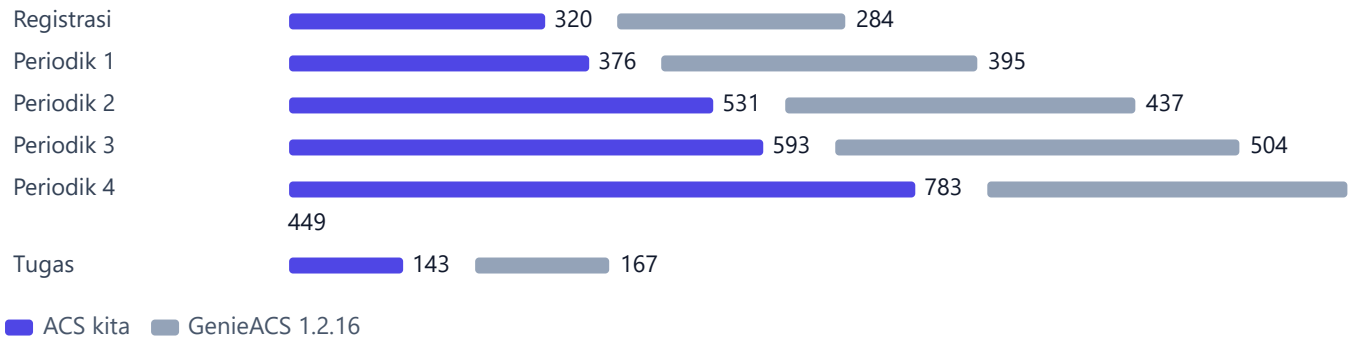
- **Latensi** = durasi satu sesi penuh (Inform → RPC → penutup) per perangkat. Persentil p50/p95/p99 dihitung dari seluruh sesi satu fase.
- **Parameter tugas** sengaja bukan yang ikut dilaporkan Inform: GenieACS menganggap nilai dari Inform masih segar dan melewati RPC, sedangkan ACS kita selalu bertanya ke CPE.
- **Sumber daya** disampel dari `docker stats` tiap 2 detik (hanya 5–6 sampel per run; 100% CPU = satu inti).

## 3. Hasil throughput dan latensi

Sesi per detik: lebih tinggi lebih baik. Latensi: lebih rendah lebih baik. Sel hijau = lebih baik pada baris itu. 1.500 sesi per fase (500 pada fase tugas); 0 galat.

| Fase | ACS kita | | | | GenieACS 1.2.16 | | | |
|---|---|---|---|---|---|---|---|---|
| | sesi/dtk | p50 | p95 | p99 | sesi/dtk | p50 | p95 | p99 |
| Registrasi | **320** | **187 ms** | **278 ms** | **309 ms** | 284 | 191 ms | 469 ms | 628 ms |
| Periodik putaran 1 | 376 | 162 ms | **231 ms** | **279 ms** | **395** | **144 ms** | 286 ms | 399 ms |
| Periodik putaran 2 | **531** | **101 ms** | **244 ms** | **287 ms** | 437 | 127 ms | 286 ms | 371 ms |
| Periodik putaran 3 | **593** | **102 ms** | **157 ms** | **181 ms** | 504 | 111 ms | 239 ms | 329 ms |
| Periodik putaran 4 | **783** | **78 ms** | **116 ms** | **137 ms** | 449 | 124 ms | 280 ms | 361 ms |
| Tugas (antre API + sesi + RPC), 500 | 143 | 436 ms | **570 ms** | **627 ms** | **167** | **356 ms** | 612 ms | 690 ms |

### Throughput (sesi/detik)




### Cara membaca

- **Registrasi dan putaran 1 (pemanasan)** setara; selisihnya masih dalam rentang kebisingan run.
- **Putaran 2–4 (kondisi tunak)**: ACS kita lebih tinggi 18–74% throughput-nya dan p95 lebih rendah 15–59%. Selisih ini searah pada ketiga putaran, tetapi GenieACS baru diukur **satu kali**; perbaikan jalur Inform di sisi kita sendiri terbukti konsisten pada run berulang (bagian 5).
- **Fase tugas**: GenieACS unggul di throughput dan p50; ACS kita unggul di p95/p99. Pada run lain hari ini fase tugas ACS kita lebih lambat dari angka ini, sehingga fase ini **belum bisa diklaim** menguntungkan sisi mana pun.

## 4. Sumber daya

| Komponen | CPU rata-rata | CPU puncak | Memori maks |
|---|---|---|---|
| **ACS kita** — proses `acsd` | 159% | 188% | **25 MiB** |
| ACS kita — MariaDB | 259% | 457% | 165 MiB |

| | | | |
|---|---|---|---|
| ACS kita — Redis | 15% | 18% | 6 MiB |
| **GenieACS** — `genieacs-cwmp` | 334% | 480% | 589 MiB |
| GenieACS — MongoDB | 174% | 249% | 416 MiB |
| GenieACS — NBI | 9% | 54% | 190 MiB |
| **Total** ACS kita / GenieACS | ≈433% / ≈517% | — | **≈196 MiB / ≈1.195 MiB** |

- Proses ACS kita memakai **±24× lebih sedikit memori** dan ±2× lebih sedikit CPU daripada proses CWMP GenieACS untuk throughput yang sama atau lebih tinggi.
- **MariaDB adalah komponen terberat** di sisi kita (rata-rata ±2,6 inti) — lebih berat daripada MongoDB (±1,7 inti). Ini beban tulis per Inform (sesi, event, audit) dan masih menjadi kandidat optimasi berikutnya.
- Sampel hanya 5–6 titik per run, jadi persentase CPU adalah perkiraan kasar.

## 5. Optimasi yang dilakukan hari ini dan buktinya

Benchmark 21 Oktober menunjukkan ACS kita setara setelah pemanasan, tetapi lebih lambat saat registrasi dan fase tugas, dengan MariaDB berat. Penelusuran log query MariaDB menemukan tiap query dikirim sebagai tiga round trip (Prepare, Execute, Close) dan hot path Inform melakukan SELECT/UPDATE berulang.

| Perubahan | Efek |
|---|---|
| Satu query = satu round trip ( `interpolateParams` otomatis pada koneksi DB) | Hilang 2 round trip per query |
| Namespace CWMP ikut `INSERT` sesi baru | −1 UPDATE, −1 SELECT, −operasi Redis per sesi baru |
| `UpsertBatch` satu chunk tanpa `BEGIN/COMMIT` | −2 round trip dan −1 commit per Inform |
| Cache 5 detik untuk aturan ZTP dan preset enforce (dibuang langsung saat diubah di instance yang sama) | −2 SELECT per Inform |
| Event status task memakai task yang sudah dipegang | −3 SELECT per tugas |

**Query per sesi** (MariaDB `Com_*`): SELECT 13,3 → 10,3 · UPDATE 3,5 → 2,5 · commit 1,5 → 0,2.

### Pembandingan sebelum/sesudah pada kode kita sendiri (beban penuh, 2 run per sisi, urutan selang-seling)

| Fase | Sebelum (sesi/dtk · p95), run 1 / run 2 | Sesudah (sesi/dtk · p95), run 1 / run 2 |
|---|---:|---:|
| Registrasi | 144 · 704 ms / 160 · 750 ms | 224 · 442 ms / 329 · 274 ms |
| Periodik putaran 4 | 310 · 342 ms / 403 · 247 ms | 636 · 144 ms / 701 · 124 ms |
| Tugas | 66 · 1.400 ms / 89 · 1.125 ms | 79 · 1.483 ms / 108 · 813 ms |

Jalur Inform ±1,7–2× lebih cepat dan p95 turun 2–3×; selisih antar-run jauh lebih kecil daripada selisih sebelum/sesudah. Fase tugas membaik tipis dan masih berisik. Verifikasi kode: `go vet` dan `go test ./...` hijau, suite e2e 30/30 lulus pada kode baru.

Kompromi: aturan ZTP/preset yang diubah lewat instance acsd lain baru terlihat setelah ≤5 detik. Lookup tenant per Inform sengaja tidak di-cache supaya penonaktifan tenant tetap langsung memutus akses.

## 6. Perbandingan fitur

Kemampuan GenieACS diverifikasi pada instans 1.2.16 yang berjalan (NBI menjawab 200 pada `/devices`, `/tasks`, `/presets`, `/provisions`, `/virtual_parameters`, `/files`, `/faults`, `/objects`).

| Kemampuan | ACS kita | GenieACS 1.2.16 |
|---|:---:|:---:|
| Multi-tenant, RBAC bertingkat, MFA TOTP, SSO OIDC | Ada | Peran dasar, tanpa tenant |
| Audit trail, notifikasi in-app/email, webhook, billing SaaS | Ada | Tidak ada |
| Tab WiFi, dashboard optik/suhu/uptime, retry task, "Refresh ONT", capture sesi dengan penyamaran rahasia | Ada | UI generik |
| Portal pelanggan akhir (self-service) | Ada | Tidak ada |
| Katalog vendor/path sebagai data; enkripsi rahasia task | Ada | Lewat skrip/provision |
| Provision berskrip (JavaScript), **virtual parameter** | Tidak ada | Ada |

| | | |
|---|---|---|
| Query perangkat bebas + proyeksi + sort | Filter tetap | Ada |
| Resource `faults` tersendiri (daftar, hitung kejadian, tutup otomatis saat RPC berhasil) | Ada (22 Okt 2026) | Ada |
| Kematangan lintas firmware dan skala produksi | Baru 3 ONT fisik | Bertahun-tahun |

## 7. Yang tidak diukur

- **Perangkat dan firmware nyata.** Uji ONT fisik 21 Oktober menemukan bug `xsi:type` yang tidak tertangkap simulator; benchmark ini tidak menggantikan uji lapangan.
- **Skala** ratusan ribu hingga jutaan perangkat, dan retensi data berhari-hari (pertumbuhan tabel, kinerja query lama).
- **Beban REST/UI bersamaan** dengan Inform, serta kegagalan jaringan dan restart komponen.
- **Pengaruh autentikasi.** ACS kita memverifikasi kredensial CWMP per tenant pada tiap Inform dan mencatat sesi, event, dan audit; GenieACS bawaan tanpa autentikasi CWMP. Sebagian selisih resource adalah harga fitur, bukan semata efisiensi.
- **GenieACS dengan tuning** (jumlah worker, indeks MongoDB, ekstensi) — yang diuji adalah konfigurasi bawaan.

## 8. Rekomendasi

1. **Jangan memakai benchmark ini sebagai klaim keandalan.** Klaim yang didukung: efisiensi memori dan throughput Inform setara-atau-lebih-baik pada beban simulator; fitur platform multi-tenant.
2. **Profil fase tugas** (antre via API: cek kuota, `activity_logs`, `INSERT tasks`) sebelum mengklaim apa pun di fase itu.
3. **Kurangi beban tulis MariaDB per Inform** (event dan audit) bila skala perangkat mendekati ratusan ribu.
4. **Ulangi di mesin khusus** (server terpisah dari generator beban, beberapa run, median) bila angka akan dipakai di luar tim.
5. **Tutup gap fitur** yang relevan bagi pelanggan: virtual parameter dan query perangkat bebas; provision berskrip perlu keputusan desain lebih dulu.

---

Sumber: `docs/pengujian/BENCHMARK_GENIEACS.md` (catatan teknis), `bin/bench-ours.json`, `bin/bench-genieacs.json`. PDF dibuat dari `BENCHMARK_GENIEACS.html` dengan `node scripts/build-docs-pdf.mjs BENCHMARK`. Sunting HTML-nya, bukan PDF.